본 자료는 저작권법 제25조 2항에 의해 보호를 받으며, **pdf 형태로 보실 수 있도록 공유드리고 있습니다.**
본 자료를 인터넷상에 오픈 또는 공유하시면, 쉽게 확인이 되어 저작권법에 의해 부득이 조치를 취할 수 밖에 없으므로, 인터넷상에 오픈 또는 공유는 하지 말아주세요
추가로 **잔재미코딩 가장 빠른 풀스택 로드맵**과 함께 익히시면, 보다 자유자재로 바이브 코딩을 익히고 활용할 수 있습니다
관련 강의도 **잔재미코딩(노이먼스랩)** 에서 확인하실 수 있으며,
해당 사이트에서 회원가입하시면, 최신 AI 및 자동화, 데이터, 풀스택 관련 트렌드 뉴스레터도 공유받으실 수 있습니다.

Claude Code 바이브 코딩: 프로젝트 빠른 시작 가이드

Claude Code 기반 바이브 코딩 단계별 로드맵

📌 웹페이지 기본 개념

정적 웹페이지 vs 동적 웹페이지

- **정적 웹페이지**: 미리 만들어진 HTML, CSS, JavaScript 파일을 그대로 보여주는 웹페이지
  - 예: 회사 소개 페이지, 포트폴리오, 제품 소개 페이지
  - 모든 사용자에게 동일한 내용 표시
  - 서버나 데이터베이스 없이 작동
- **동적 웹페이지**: 사용자의 요청이나 데이터에 따라 내용이 실시간으로 바뀌는 웹페이지
  - 예: SNS 피드, 쇼핑몰, 블로그, 게시판
  - 사용자마다 다른 내용 표시 (로그인한 사람, 구매 내역 등)
  - 백엔드 서버와 데이터베이스 필요

1단계: 정적 웹페이지 (고정된 웹페이지)

> 데이터를 별도 저장하거나, 관리할 필요 없고, 로그인/회원관리도 필요없이 화면에만 보여주면 되는 웹페이지

💡 바로 Claude Code로 웹개발 해보기 - 입문자도 가능한 바이브코딩

기술 스택:

- HTML (웹페이지 구조), CSS (디자인/스타일), JavaScript (동작/인터랙션) + Tailwind CSS (최신 디자인 스타일)
- GitHub Pages (무료 호스팅 서비스)

특징:

- AI 활용시 10 분만에도 개발 가능 (배포도 10분만에 가능)
- 포트폴리오, 랜딩 페이지 (제품 소개 페이지), 간단한 정보 사이트에 적합
- **IT 지식 없이도 Claude Code만으로 평생 가능**

2단계: 간단한 데이터 관리 기능을 사용하는 웹서비스 (회원관리/로그인 미포함)

> 데이터 저장, 관리등은 필요하지만, 로그인등 회원관리는 불필요한 웹서비스
>
> - 로그인등이 들어가면 회원관리 관련 메뉴부터 시작해서, 회원별 데이터 관리 및 강력한 보안이 필요함
> - 데이터 관리 및 저장이 필요한 경우 (데이터 추가,읽기,수정,삭제 (CRUD))
> - 이외의 기능 필요시에는 3단계를 추천

🔥 좀 더 복잡한 웹개발과 Claude Code - 입문자도 연습하면 가능한 바이브코딩

기술 스택:

- **Backend (백엔드)**: Javascript (Cloudflare Pages Functions 으로 배포)
- **Frontend (프론트엔드)**: HTML, CSS, JavaScript + Tailwind CSS (Cloudflare Pages 으로 배포)
- **DB (데이터베이스)**: Cloudflare D1 (SQL을 지원하는 데이터베이스)
- **Deploy (배포)**: Cloudflare Pages + Pages Functions (서버리스 플랫폼 = 서버 관리 불필요)

특징:

- Cloudflare Pages Functions 는 Cloudflare Pages 와 연결되어 AI 가 연결을 실수할 확률이 적음
- Cloudflare Pages Functions 는 Cloudflare Workers 라는 서버리스 백엔드 서비스를 사용하며, Cloudflare Workers 라는 서비스는 파이썬을 지원하지만, 파이썬이 2024년부터 지원되어, AI 가 Javascript 만을 지원한다고 오해하는 경우가 있으므로, 그냥 Javascript 를 사용하는 것을 추천
- 서버리스 환경은 서버가 없는 환경으로 간단한 CRUD (생성/읽기/수정/삭제) 기능 외에 다양한 기능을 지원하지는 못함 (이를 위해서는 전문 프레임워크를 사용하여, 다양한 기능을 지원할 수 있는 서버를 사용해야 하므로 3단계를 추천)

왜 이 방식이 좋은가?

- Cloudflare의 글로벌 CDN으로 빠른 속도 + 무료 사용 한도가 충분함
- Cloudflare MCP 로 Cloudflare 를 몰라도 배포 가능
- Cloudflare Pages + Pages Functions 으로 연결되어 있어서, AI 가 통상적으로 실수하는 다양한 프론트엔드와 백엔드 연결 문제를 막을 수 있음

3단계: 복잡한 백엔드 (회원관리 포함) + DB 사용 웹서비스

> - 데이터 저장,관리 및 로그인 포함 회원관리, 보안처리까지 하려면 여러 개발자가 필요하지만, **효율적인 기술셋을 사용하여 혼자서도 개발 가능**하고, **보안등 상시적으로 엄격한 관리가 필요한 부분은 Supabase 라는 검증된 서비스를 활용**하여, 결국 **혼자서도 어떤 상용 서비스도 개발 가능**
> - 단, 개발 복잡도가 높아서 개발 단계에서 다양한 상황에 대응해야 하고, 복잡한 기능으로 개인 PC 상에서 테스트 환경을 구축하고, 테스트해야 하며 필요시 AI 와 코드 레벨 커뮤니케이션이 필요함
> - 따라서 3단계에서는 공유드리는 **풀스택 로드맵을 익히며, 꾸준히 연습하는 것을 추천드림, 연습하면 할 수록 개발가능범위가 늘어남!**

🚀 IT 기본기를 쌓으면서 연습하면 어떤 웹개발도 가능한 고급 바이브 코딩

기술 스택:

- 풀스택 : FastAPI + Jinja2 Template (HTML, CSS, JS + HTMX + Tailwind CSS)
- Database (데이터베이스): Supabase (PostgreSQL 기반 클라우드 DB)
- Deploy (배포): AWS EC2 (클라우드 서버)
  - 완전한 서버 제어가 가능하며, 전문 백엔드 프레임워크를 사용, 상용 수준의 서비스 구축 가능
  - Docker 배포 기술 활용
- 인증: Supabase Auth (사용자 로그인 시스템, FastAPI를 통해 프록시)

특징:

- **프로덕션 수준 (실제 서비스 가능)의 실전 프로젝트 배포 가능**
- 효율적인 기술셋 기반 최대한 간결한 코드로 최신 웹서비스 구현 가능
- 상용 서비스 수준의 보안 및 사용자 인증 (로그인/회원가입) 기능 지원 가능
- 확장 가능한 아키텍처 (사용자 증가에 대응도 가능)

옵션: 화려한 프론트엔드 (현대적 SPA)

⭐ React/Next.js 프레임워크 경험자를 위한 현대적 풀스택 개발

왜 프론트엔드 프레임워크를 사용하는가?

다양한 화면이 수시로 바뀌는 복잡한 웹앱의 경우:

- **순수 JavaScript 문제점**: 성능이 좋지 않은 컴퓨터에서 화면이 깜빡이거나 느릴 수 있음
- **React의 장점**: 화면 업데이트를 효율적으로 처리하여 부드러운 사용자 경험 제공
- **React의 단점**: SEO (구글 검색 최적화)가 애매함 → 검색 엔진이 내용을 제대로 읽지 못할 수 있음
- **Next.js의 해결책**: React의 장점 + SEO도 가능 (서버에서 미리 HTML 생성)

⚠️ **하지만 학습 난이도가 높습니다:**

- HTML, CSS, JavaScript는 기본이고, React, Next.js, TypeScript, 상태 관리, 라우팅 등 여러 기술이 복잡하게 엮여 있음
- 바이브 코딩으로 빠르게 만들 수는 있지만, 코드를 이해하고 수정하려면 관련 기술 학습이 필요
- **추천 대상: 관련 기술을 어느 정도 익힌 분들**

기술 스택:

- Frontend (프론트엔드): Next.js (React 기반 프레임워크 with App Router)
- Backend (백엔드): FastAPI (Python 웹 프레임워크)
  - 백엔드를 Next.js 프레임워크 자체적으로도 구현 가능하지만, 파이썬이 가장 다양한 기능을 지원하므로, FastAPI 도 추천함
- Database (데이터베이스): Supabase (PostgreSQL 기반)
- Deploy (배포): AWS EC2 (프론트엔드 및 백엔드)
  - Docker 배포 기술 활용

대상:

- React/Next.js 경험자 또는 학습 중인 분
- 복잡한 UI/UX (사용자 인터페이스/경험)가 필요하여, 웹페이지 성능도 중요한 프로젝트
- 검색 엔진 최적화 (SEO)가 중요한 서비스

단계별 선택 가이드

1단계를 선택하세요:

- ✅ 백엔드 (서버)가 필요 없는 정적 사이트 (고정된 웹페이지)
- ✅ 빠른 프로토타이핑 (시제품 제작)
- ✅ 포트폴리오, 랜딩 페이지 (제품 소개 페이지)
- ❌ 사용자 로그인 필요
- ❌ 데이터베이스 필요

2단계를 선택하세요:

- ✅ 간단한 CRUD (생성/읽기/수정/삭제) 앱
- ✅ 서버 관리 부담 제로 (서버리스 = 서버 관리 불필요)
- ✅ 빠른 배포 필요

- ✅ Cloudflare의 무료 티어 (무료 사용 한도)로 충분히 사용 가능
- ⚠️ 복잡한 인증 (로그인)/권한은 3단계 권장

3단계를 선택하세요:

- ✅ 완전한 사용자 인증 (로그인)/권한 관리
- ✅ 복잡한 비즈니스 로직 (업무 규칙)
- ✅ 프로덕션 환경 (실제 서비스 운영)
- ✅ 보안이 중요
- ⚠️ 서버 관리 및 관련 사용법 숙지 필요 (AWS)
- ⚠️ 비용 발생

옵션을 선택하세요:

- ✅ 복잡한 UI/UX (사용자 인터페이스/경험)가 필요하여, 웹페이지 성능도 중요
- ✅ React/Next.js 를 익힌 분 대상
- ⚠️ 개발자 레벨로 익혀야 함
- ⚠️ 코드 이해도 필요

CLAUDE.md 작성법

CLAUDE.md는 Claude Code의 영구 메모리 (프로젝트 정보 저장소)로, 프로젝트 루트 (최상위 폴더)에 **100줄 이내**로 간결하게 작성하세요.

기본 구조

```
# [프로젝트명]

## 목적
1-2줄로 설명

## 기술 스택 (사용할 기술셋 기술)
- 프론트엔드: HTML, CSS, JavaScript + Tailwind CSS
- 배포: GitHub Pages

## 핵심 규칙 (개발할 때 지켜야 할 원칙)
- 필요시

## 프로젝트 구조
```

```
project/
├── backend/  # 백엔드
├── frontend/ # 프론트엔드
└── designs/  # 디자인 파일
```

작성 원칙: 간결하게, 핵심만, 구조화

단계별 간략 템플릿

1단계: 정적 웹사이트

```
# 프로젝트: 이력서 웹페이지

## 목적
개인용 이력서 웹페이지

## 기술 스택
- 프론트엔드: HTML, CSS, JavaScript + Tailwind CSS
- 배포: GitHub Pages

## 핵심 규칙
- Apple 디자인 스타일을 따라주세요

## 구조
project/
├── index.html
├── css/
└── js/
```

2단계: Cloudflare

```
# 프로젝트: MyTODO 웹서비스

## 목적
개인용 할일 저장 및 관리 웹서비스

## 기술 스택
- 프론트엔드: HTML, CSS, JS + Tailwind CSS (정적 파일)
```

```
- 백엔드: JavaScript (Cloudflare Pages Functions 용)
- 데이터베이스: Cloudflare D1
- 배포: Cloudflare Pages + Cloudflare Pages Functions

## 핵심 규칙
- 로그인 기능 불필요
- 토스 디자인 스타일을 따라주세요

## 구조
todo-app/
├── public/          # 정적 파일 (Cloudflare Pages)
├── functions/       # 백엔드 (Cloudflare Pages Functions)
└── designs/         # XML 디자인 파일
```

3단계: FastAPI + Supabase

```
# 프로젝트: 블로그 웹서비스

## 목적
블로그 웹서비스

## 기술 스택
- 풀스택: FastAPI + Jinja Template(HTML, CSS, JS + Tailwind CSS + HTMX)
- 데이터베이스: Supabase
- 배포: AWS EC2 (도커로 작성하여 배포 예정)

## 보안 규칙
- Supabase 키는 FastAPI 백엔드에서만 사용하고, 클라이언트에서는 사용 금지
- 뱅크샐러드 디자인 스타일을 따라주세요

## 구조
blog/
├── main.py          # FastAPI 앱
├── templates/       # Jinja 템플릿
├── static/          # CSS, JS
└── designs/         # XML 디자인 파일
```

XML로 화면 디자인하기

XML (구조화된 텍스트 형식)로 디자인을 작성하면 색상, 크기, 배치를 AI에게 명확히 전달할 수 있습니다.

1단계: 랜딩 페이지 예시

```
<화면 이름="제품소개페이지">
  <히어로섹션 배경색="연한회색" 높이="높게">
    <제목 크기="매우크게" 색상="검정">우리 제품을 소개합니다</제목>
    <설명 크기="보통" 색상="회색">혁신적인 솔루션으로 여러분의 업무를 돕습니다</설명>
    <버튼 배경색="파란색" 색상="흰색">시작하기</버튼>
  </히어로섹션>

  <기능소개 배경색="흰색">
    <기능 아이콘="✓" 제목="빠른 속도" 설명="초고속 처리"></기능>
    <기능 아이콘="✓" 제목="쉬운 사용" 설명="직관적 인터페이스"></기능>
    <기능 아이콘="✓" 제목="안전함" 설명="강력한 보안"></기능>
  </기능소개>
</화면>
```

2단계: TODO 앱 예시

```
<화면 이름="myTODO">
  <제목 크기="크게">할 일 목록</제목>

  <입력영역>
    <입력창 힌트="할 일 입력"></입력창>
    <버튼>추가</버튼>
  </입력영역>

  <할일목록>
    <할일항목>
      <할일>장보기</할일>
      <버튼>수정</버튼>
      <버튼>삭제</버튼>
    </할일항목>
  </할일목록>
</화면>
```

3단계: 블로그 로그인 화면 예시

```
<화면 이름="로그인">
  <로그인폼 최대너비="중간" 배경색="흰색">
    <제목 크기="크게" 색상="검정">로그인</제목>

    <입력그룹>
      <레이블>이메일</레이블>
      <입력창 힌트="your@email.com"></입력창>
    </입력그룹>

    <입력그룹>
      <레이블>비밀번호</레이블>
      <입력창 힌트="비밀번호"></입력창>
    </입력그룹>

    <버튼 배경색="초록색" 색상="흰색" 너비="전체">로그인</버튼>
    <링크 색상="파란색">회원가입</링크>
  </로그인폼>
</화면>
```

AI에게 적용 요청

```
> @designs/main.xml 참고해서 HTML/CSS 만들어줘
```

💡 **팁:** AI에게 XML 초안 작성 맡기기

```
> TODO 앱 메인 화면을 XML로 디자인해줘. designs/main.xml로 저장
```

프로젝트 시작 간단 워크플로우

모든 프로젝트 공통

```
# 1. 프로젝트 폴더 생성
mkdir my-project && cd my-project

# 2. CLAUDE.md 작성 (프로젝트 정보 문서, 또는 /init으로 자동 생성)
# 3. (선택) designs/ 폴더에 XML 디자인 작성 (화면 구조 설계)
# 4. Claude Code 실행
claude --dangerously-skip-permissions

# 5. AI에게 요청
> @CLAUDE.md 참고해서 프로젝트 초기 구조 만들어줘
```

마무리

이제 단계별로 프로젝트를 시작할 준비가 되었습니다!

시작 체크리스트

모든 단계 공통:

- [ ] CLAUDE.md 작성 (프로젝트 목적, 기술 스택, 규칙)
- [ ] 디렉토리 구조 (폴더 구조) 생성
- [ ] Claude Code 실행 (`claude` 또는 `claude --dangerously-skip-permissions`)

디자인이 중요한 프로젝트:

- [ ] designs/ 폴더 생성
- [ ] XML 디자인 파일 작성 (화면 구조, AI에게 맡겨도 됨)
- [ ] AI에게 XML 참조하여 코드 생성 요청

2단계 프로젝트 (FastAPI + Cloudflare):

- [ ] backend/ 폴더 생성 (FastAPI 앱 = 백엔드 API)
- [ ] frontend/ 폴더 생성 (HTML, CSS, JS = 프론트엔드)
- [ ] Cloudflare MCP 설정 (Workers + Pages 배포용)

3단계 프로젝트 (프로덕션):

- [ ] backend/app/ 폴더 구조 생성
- [ ] Supabase 프로젝트 생성 및 MCP 설정
- [ ] 보안 설정 (API 키 숨기기, CORS = 접근 제어 등)

- [ ] AWS EC2 구축

꿀팁:

1. **작은 것부터 시작**: 1단계로 프로토타입 (시제품) → 2-3단계로 확장
2. **XML 활용**: 디자인을 명확히 전달 → 결과물 품질 향상
3. **Plan Mode (계획 모드)**: 복잡한 프로젝트는 Shift+Tab으로 계획 먼저
4. **YOLO 모드 (자동 승인 모드)**: 학습 단계에서는 `--dangerously-skip-permissions` 로 빠르게
5. **Git 사용 (버전 관리)**: 중요한 프로젝트는 GitHub Desktop 또는 GitHub MCP 로 주요 버전 저장 추천

본 자료는 저작권법 제25조 2항에 의해 보호를 받으며, **pdf 형태로 보실 수 있도록 공유드리고 있습니다.**
본 자료를 인터넷상에 오픈 또는 공유하시면, 쉽게 확인이 되어 저작권법에 의해 부득이 조치를 취할 수 밖에 없으므로, 인터넷상에 오픈 또는 공유는 하지 말아주세요
추가로 **잔재미코딩 가장 빠른 풀스택 로드맵**과 함께 익히시면, 보다 자유자재로 바이브 코딩을 익히고 활용할 수 있습니다
관련 강의도 **잔재미코딩(노이먼스랩)** 에서 확인하실 수 있으며,
해당 사이트에서 회원가입하시면, 최신 AI 및 자동화, 데이터, 풀스택 관련 트렌드 뉴스레터도 공유받으실 수 있습니다.